

Módulo 6: Librería estándar y E/S

Contenidos clave

Pathlib y manejo de archivos: pathlib sirve para trabajar con rutas y archivos de una manera más segura y sencilla que manipular las rutas como simples cadenas, es decir sin depender de rutas escritas manualmente como strings.

CSV/JSON/YAML: parseo y serialización: CSV es muy común para información tabular. JSON es muy utilizado en APIs y se puede convertir a un diccionario en Python y viceversa. JSON -> Python - deserialización.

Python -> JSON - serialización.

YAML también sirve para representar datos estructurados y es muy utilizado para archivos de configuración.

Datetime y zonas horarias: datetime permite trabajar con fechas y horas, también se puede calcular diferencias. Las zonas horarias son importantes en aplicaciones que trabajan internacionalmente.

Logging y configuración: logging sirve para registrar lo qué está ocurriendo en nuestro programa, en aplicaciones reales. Se pueden tener diferentes niveles:

DEBUG -> INFO -> WARNING -> ERROR -> CRITICAL.

Permite distinguir niveles de importancia, guardar mensajes en archivos, incluir fecha/hora, identificar de qué módulo proviene el mensaje y controlar que mensajes se quieren mostrar. Es la forma profesional de registrar eventos y errores de una aplicación.

Subprocess y automatización : subprocess permite que Python ejecute comandos o programas externos; sirve para automatización. Ejemplo: Python -> ejecuta Git -> ejecuta una herramienta -> obtiene resultado -> continúa el proceso.

Objetivos

- Manipular archivos/rutas de forma segura.

Utilización de pathlib para la manipulación de rutas y archivos de forma segura.

```
9 from pathlib import Path
```

Construcción de las rutas:

```
192 base_dir = Path(__file__).parent / "data"
193 input_file = base_dir / "raw_sales.csv"
194 output_file = base_dir / "metrics_report.json"
```

Verificación de que el archivo existe:

```
76 if not file_path.exists():
```

Creación de directorios necesarios:

```
167 output_path.parent.mkdir(parents=True, exist_ok=True)
```

Archivo de entrada:

```
178 csv_path.parent.mkdir(parents=True, exist_ok=True)
```

- Configurar logging estructurado.

Inicia con la configuración:

```
13 LOGGING_CONFIG: dict[str, Any] = {  
41 logging.config.dictConfig(LOGGING_CONFIG)
```

Se tienen 2 destinos:

- Logging -> Consola -> Info. Nivel "INFO"
- Logging -> Archivo -> Debug. Nivel "DEBUG". El archivo guarda información más detallada que la consola.

Tiene diferentes niveles:

- INFO: Para eventos normales del programa.

```
81 logger.info("Iniciando la lectura del archivo CSV: %s", file_path.name)  
  
102 logger.info(  
103     "Ingesta completada. %d transacciones válidas procesadas.",  
104     len(transactions),  
105 )  
  
163 logger.info("Exportando reporte de métricas a: %s", output_path.resolve())
```

- DEBUG: Para información más detallada, útil durante el desarrollo.

```
114 logger.debug("Calculando métricas para %d transacciones...", len(transactions))  
188 logger.debug("Archivo de prueba generado en: %s", csv_path)
```

- WARNING: Para el registro de advertencias.

```
95 logger.warning(  
96     "Fila %d omitida por formato inválido: %s. Error: %s",  
97     row_num,  
98     row,  
99     err,  
100 )
```

- ERROR: Para problemas que impiden continuar normalmente.

```
77 logger.error("El archivo especificado no existe: %s", file_path.resolve())
```

Laboratorio

- Ingesta de CSV, métricas y exportación a JSON; logging con distintos niveles .

Ingesta de CSV:

```
84 reader = csv.DictReader(f)
85 for row_num, row in enumerate(reader, start=2):
```

Cada fila del CSV se transforma en un objeto.

```
87 tx = Transaction(
88     transaction_id=row["transaction_id"],
89     product_category=row["category"],
90     amount=float(row["amount"]),
91     timestamp=datetime.fromisoformat(row["timestamp"]),
92 )
```

Métricas:

La clase, se encarga de calcular las métricas.

```
109 class MetricsCalculator:
```

Obtiene ingresos totales, número de transacciones y métricas por categoría:

```
118 generated_at=datetime.now(UTC).isoformat(),
119 total_revenue=0.0,
120 total_transactions=0,
121 categories=[],
```

Exportación a JSON:

La clase:

```
158 class JSONExporter:
```

Primero convierte los dataclasses a un diccionario:

```
164 report_dict = asdict(report)
```

Y después lo convierte a JSON:

```
170 json.dump(report_dict, f, indent=2, ensure_ascii=False)
```

Archivos creados:



